

Unidad 2
Clase 6

Distintos modelos NoSQL

CLAVE–VALOR Y PROCESAMIENTO DE ALTA PERFORMANCE

Analizar el modelo clave–valor y aplicar estructuras básicas de Redis para resolver escenarios de alto rendimiento, gestión de estado y reglas temporales en procesos automatizados de análisis y control.

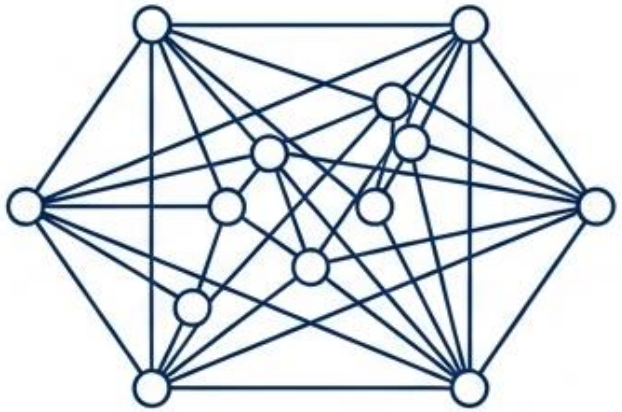
UADE

2026

The High-Speed Switchboard

Por qué existe el modelo clave-valor

Bases Relacionales: Riqueza de Información



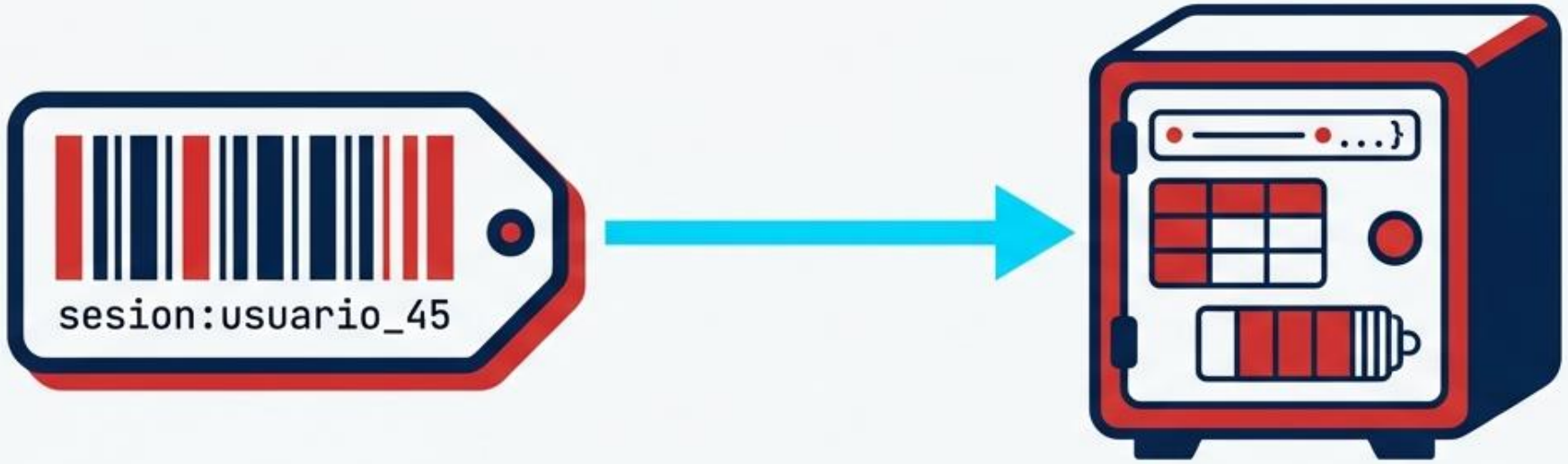
- Consultas complejas e históricas.
- Foco en la normalización y relaciones.
- El motor 'piensa' y filtra a través del universo de datos.

Clave-Valor: Eficiencia Radical



- Respuesta instantánea (milisegundos).
- Foco en el presente operativo y transitorio.
- Velocidad pura sobre consultas complejas.

Qué es una clave y qué es un valor



La Clave (Identificador)

- Referencia única y absoluta.
- Irrepetible en su espacio lógico.
- Nomenclatura semántica (ej. entidad:atributo:id).

El Valor (Contenido)

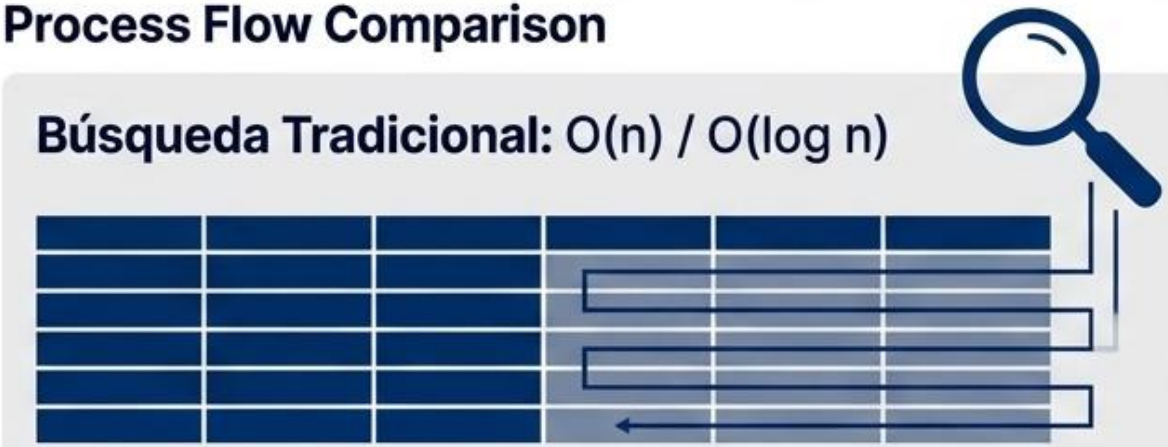
- La información atada a la clave.
- Texto, números, listas o hashes.
- Recuperación instantánea sin esquemas rígidos globales.

Analogía: Un inmenso depósito con casilleros. La clave es la etiqueta pegada afuera. Al leerla, vas directo a ese cajón sin revisar los demás.

Cómo funciona el acceso directo por clave

Process Flow Comparison

Búsqueda Tradicional: O(n) / O(log n)



- Requiere explorar, filtrar y analizar el universo de datos.
- Pregunta típica: “¿Qué registros cumplen la condición X?”

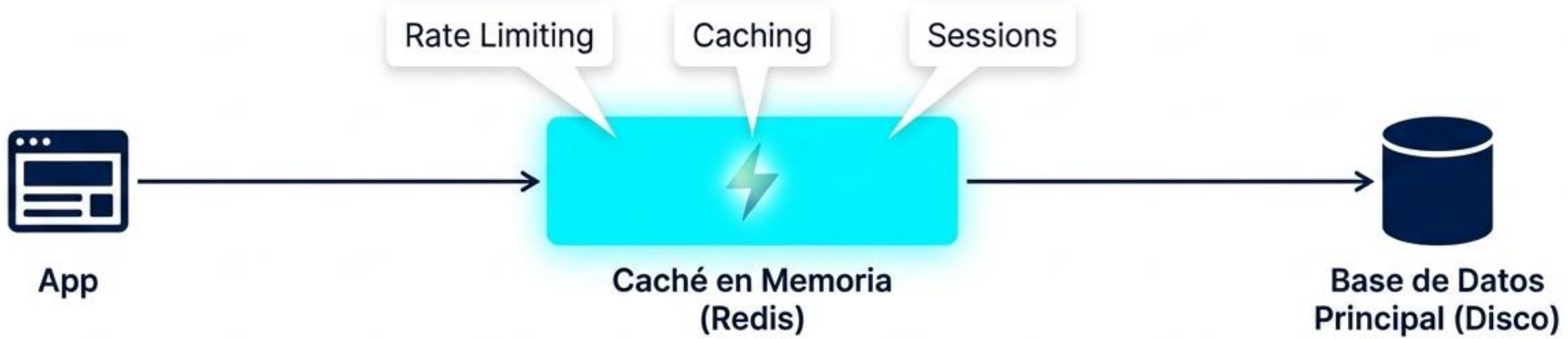
Acceso Clave-Valor: O(1)



- Funciona como un mapa de memoria (tabla hash).
- El motor no calcula ni filtra; actúa como un puntero exacto.
- Pregunta típica: “Devuélveme el valor de la clave Y, ahora.”

La Regla de Oro: Este modelo requiere que la aplicación ya sepa exactamente qué está buscando (la clave) antes de hacer la consulta.

Por qué sirve en sistemas de alto rendimiento



Operaciones In-Memory

Los datos activos residen en la RAM, eliminando el cuello de botella del disco duro. Latencias de menos de un milisegundo.

Gestión Transitoria (TTL)

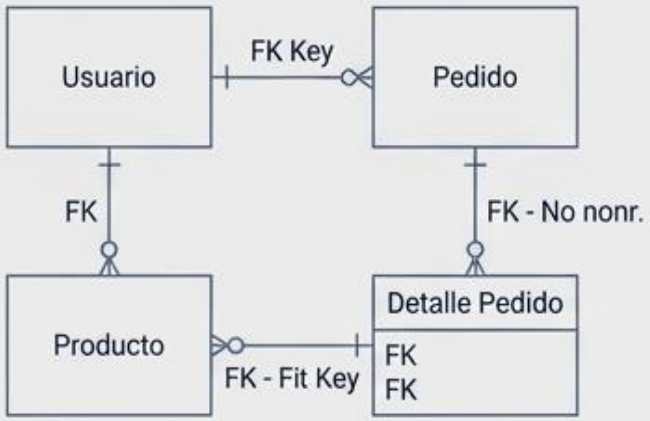
Time-To-Live. La información tiene fecha de caducidad. Los estados operativos se auto-eliminan cuando ya no son útiles.

Escudo Térmico

Descarga de trabajo pesado. Absorbe millones de lecturas/escrituras rápidas, protegiendo a las bases relacionales de la sobrecarga.

Qué cambio de mentalidad exige este modelo

Diseño Tradicional (Foco en la Entidad)



- Modelamos para representar **todo el universo del negocio**.
- **Prioridad: Evitar redundancias** (normalización).
- Diseñamos la entidad, luego vemos cómo consultarla.

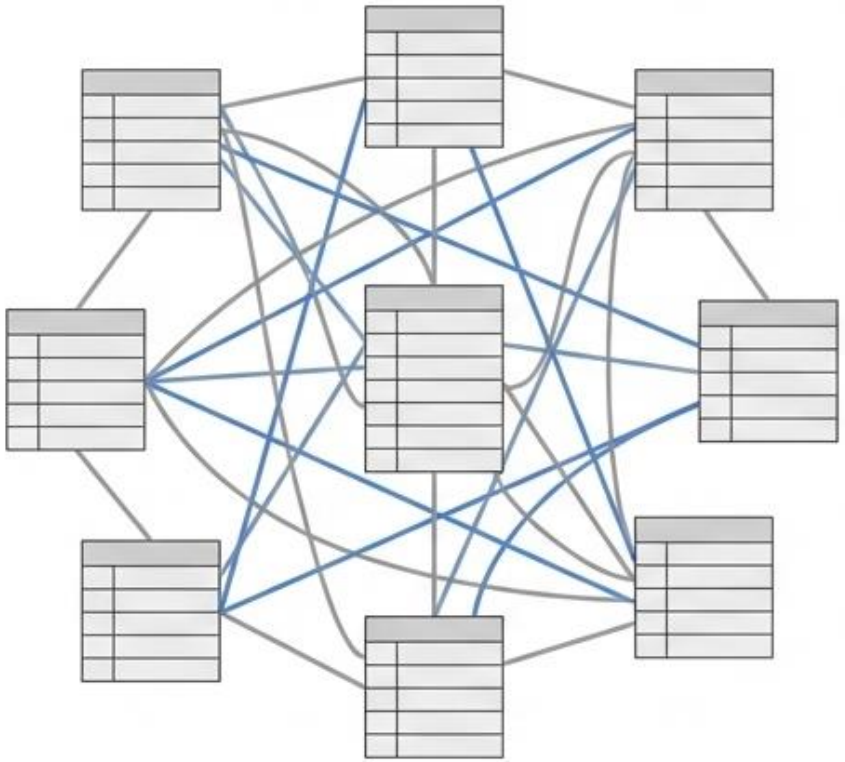
Diseño Clave-Valor (Foco en el Acceso)



- Modelamos las preguntas exactas que hará la aplicación.
- **Prioridad: Velocidad de lectura** (desnormalización estratégica).
- Diseñamos el patrón de acceso; la clave se vuelve nuestro esquema.

El mayor error en Ingeniería de Datos es intentar usar un almacén clave-valor como si fuera una base relaccional. Aquí, el dato operativo es inmensamente valioso, pero efímero.

Del Modelo Relacional al Acceso Inmediato: El Paradigma Clave-Valor



Modelo Relacional
(Búsqueda Compleja)



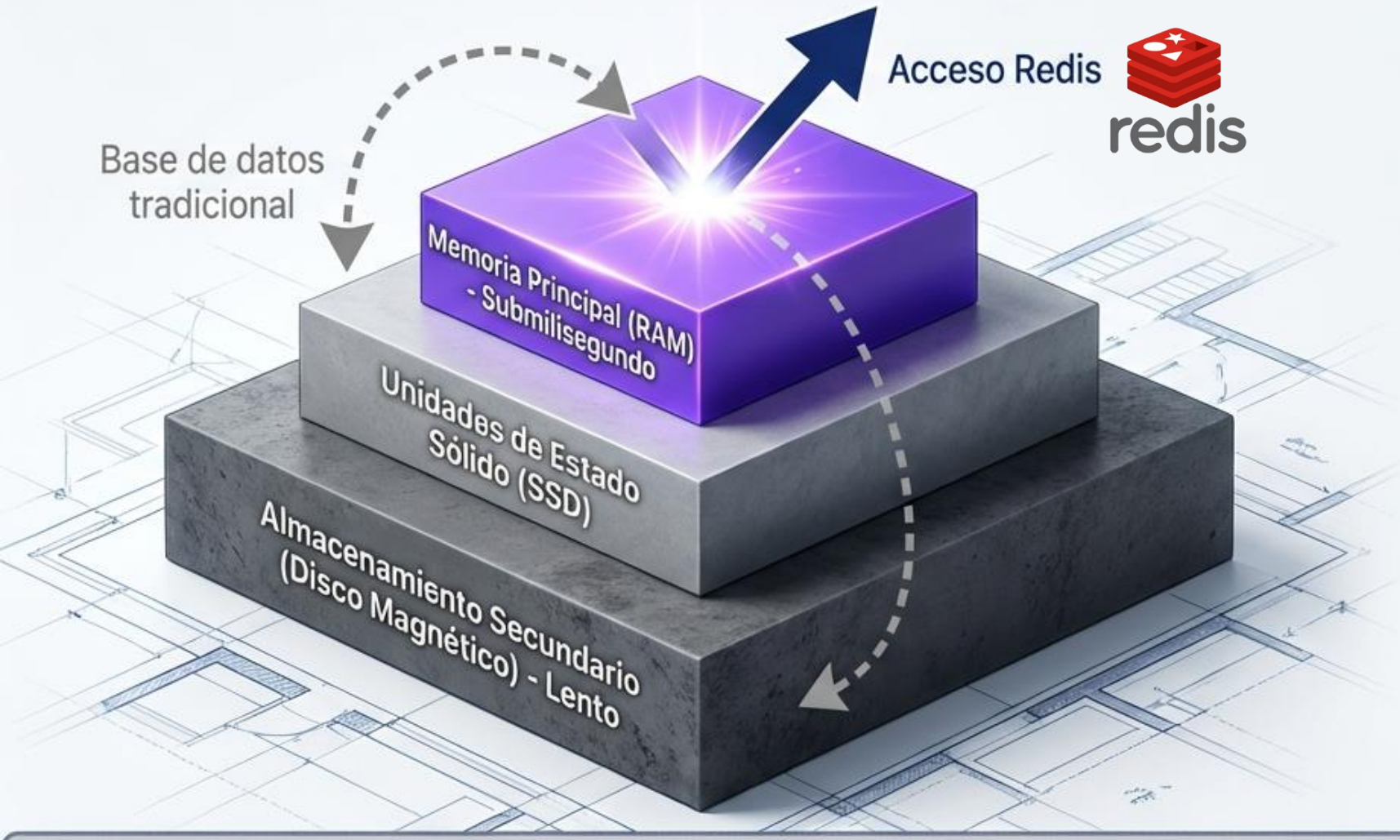
Clave-Valor
(Acceso Inmediato)

- **Cambio de enfoque:** De modelar el universo del negocio a diseñar patrones de acceso operativos.
- **Identificación directa:** Privilegia la velocidad extrema mediante una clave única conocida.
- **Propósito principal:** Resolver la latencia cuando el patrón de consulta está claro desde el inicio.

Caso Aplicado:
Un sistema no escanea 10 millones de usuarios; pregunta directo por la clave: “proveedor:203:observado”.

UADE

Arquitectura Física: El Poder de Operar en Memoria RAM



- **Datos activos en RAM:** Evita el costoso viaje de I/O al almacenamiento secundario.
- **Estructuras nativas:** Representación simple y optimizada para manipulación directa en memoria.
- **Rol arquitectónico:** Complementa a las bases tradicionales gestionando el estado operativo actual.

Caso Aplicado: Mantener la sesión bancaria de un usuario activa en RAM para validar clics en micro-oisegundos sin tocar la base central.

Clase Magistral: ¿Por qué Redis es tan rápido? Porque elimina el cuello de botella físico más grande: el disco. Al trabajar en memoria, las estructuras se manipulan directamente sin procesos de serialización. Redis es la capa de alta velocidad que sostiene el estado operativo de su arquitectura.

Rendimiento Submilisegundo y Arquitectura Single-Thread



- **Procesamiento Single-Thread:** Ejecución secuencial que elimina la complejidad y los bloqueos (locks) por concurrencia.
- **Operaciones Atómicas:** Cada comando se ejecuta por completo, de principio a fin, antes de pasar al siguiente.
- **Latencia submilisegundo:** Capacidad para procesar millones de operaciones por segundo sin cuellos de botella de software.

Caso Aplicado: Cientos de robots actualizando un contador de anomalías al mismo tiempo; Redis garantiza que ninguna suma colisione o se pierda.

Clase Magistral:

A diferencia de otros motores que abren múltiples hilos, Redis utiliza una arquitectura de un solo hilo. Al evitar los bloqueos de concurrencia y los cambios de contexto en la CPU, procesa las operaciones de manera atómica y secuencial a una velocidad asombrosa. Cada operación es un bloque indivisible.

Persistencia: Uso Transitorio vs. Conservación de Datos

	RDB (Snapshots)	AOF (Journaling)	Sin Persistencia
Mecanismo	Captura periódica del estado de la memoria	Bitácora continua de cada comando de escritura	Caché puro y volátil
Recuperación y Tamaño	Archivos compactos, reinicio muy rápido	Archivos pesados, reconstrucción precisa	Los datos se pierden al reiniciar
Riesgo de Pérdida	Minutos de datos recientes	Segundos (o cero, según configuración)	Totalidad de los datos

- **Mito roto:** Que Redis trabaje en memoria no significa que sea amnésico.
- **Estrategia híbrida:** Configurable según el valor de negocio del dato (efímero vs. crítico).

Caso Aplicado: Un carrito de compras usa persistencia AOF para no perder ventas si el servidor cae; un caché de catálogo no usa persistencia.

Clase Magistral:
Redis ofrece dos motores: RDB, que toma "fotos" eficientes con riesgo de perder los últimos minutos; y AOF, un diario de cada comando que prioriza la durabilidad. En diseño arquitectónico, la elección de persistencia es siempre una decisión de negocio.

El Ciclo de Vida del Dato: SET, GET, EXISTS, DEL

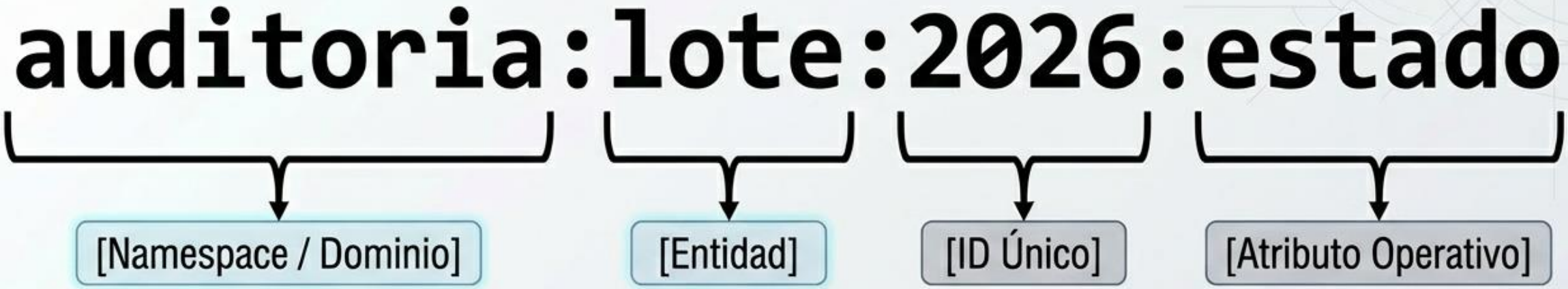
- **EXISTS**: Verifica presencia. La puerta de entrada para toda lógica condicional rápida.
- **SET**: Asigna o sobrescribe un estado operativo.
- **GET**: Recuperación inmediata del valor (unidad mínima de trabajo).
- **DEL**: Limpieza activa del estado cuando el proceso finaliza.



Caso Aplicado:
Control de tarea: EXISTS tarea_01 -> No. SET tarea_01 'ejecutando'. Al terminar la rutina -> DEL tarea_01.

Clase Magistral:
No memoricen comandos; entiendan el flujo lógico. Antes de calcular algo pesado, un sistema pregunta EXISTS. Si no existe, lo calcula y hace SET. Si existe, hace un GET inmediato. Y cuando la tarea termina, usa DEL para limpiar. Esta es la espina dorsal del manejo de estados transitorios.

Anatomía de la Clave: Diseño por Patrones de Acceso



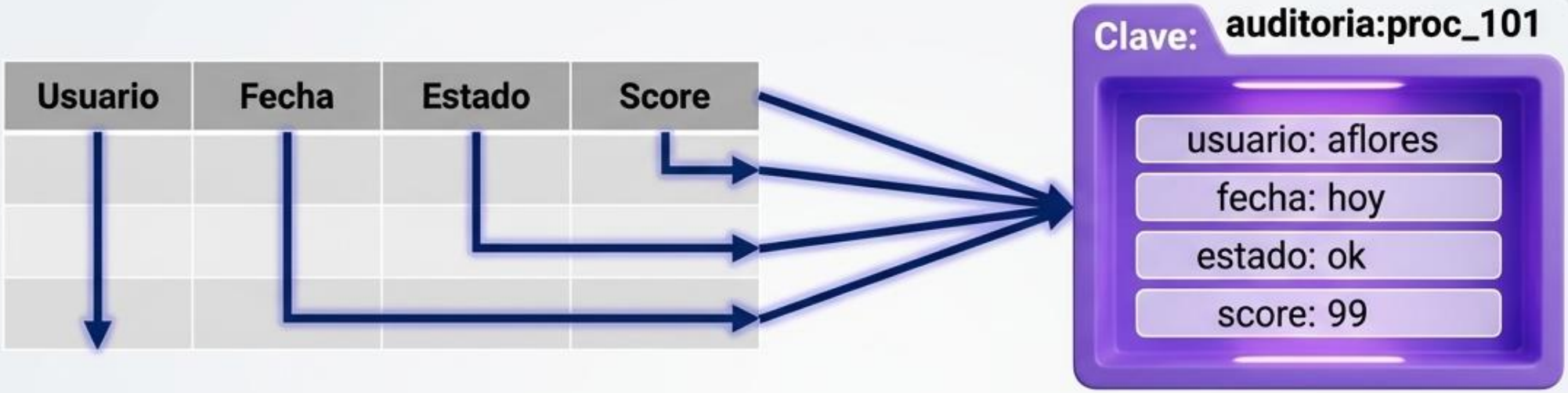
- **Diseño semántico**: La clave debe comunicar su propósito y contexto sin ninguna ambigüedad.
- **Namespaces**: Uso de separadores (dos puntos ':') para organizar jerárquicamente el espacio de memoria.
- **Antipatrón**: Evitar búsquedas completas (KEYS *); diseñar la clave para consultar el identificador exacto de forma directa.

Caso Aplicado:
En lugar de la clave opaca 'x193_status', la buena práctica es la estructura semántica 'login:intentos:usuario_45'.

Clase Magistral:
En Redis, el diseño de la base de datos ES el diseño de las claves. Al no tener tablas relacionales, la semántica recae en cómo nombramos la información. Diseñar bien una clave es diseñar bien el acceso: si el sistema necesita el estado del lote 2026, va directo a su clave sin escanear nada más.

Hashes: Agrupación Lógica de Entidades

UADE



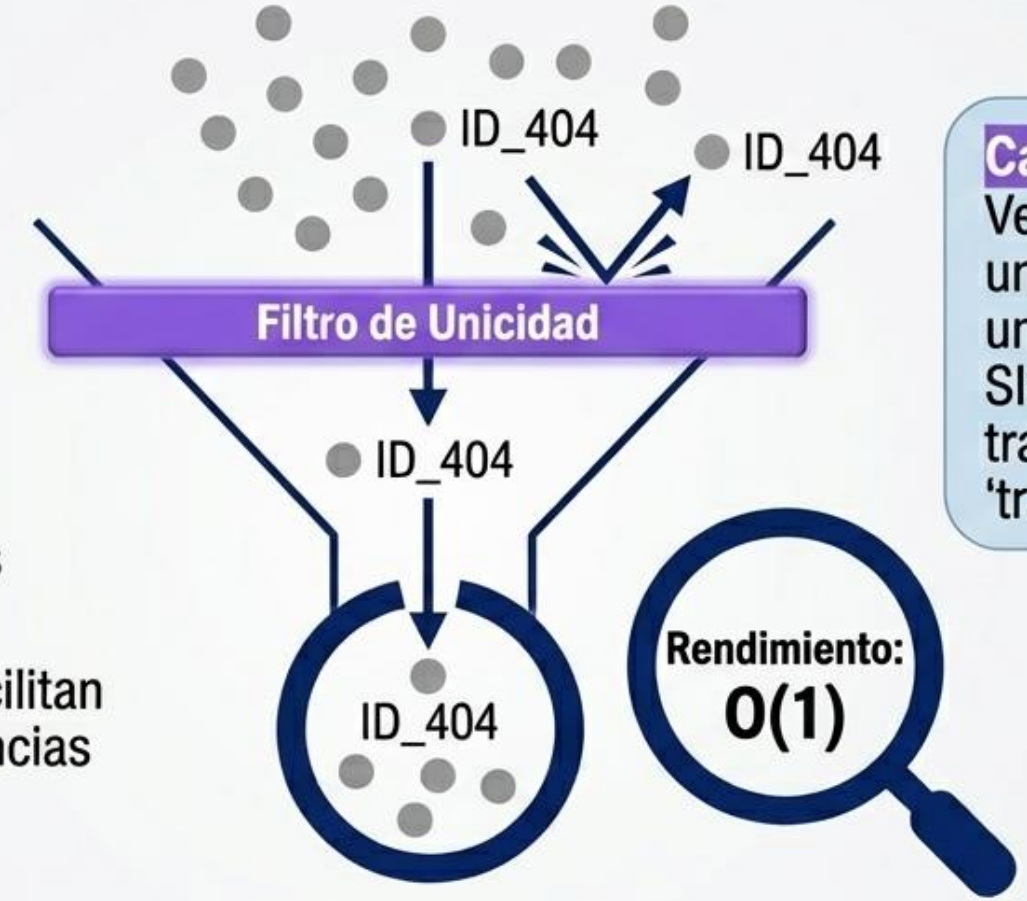
- **Estructura tipo registro:** Pares campo-valor ideales para modelar objetos básicos de negocio en una sola lectura.
- **Organización eficiente:** Evita la proliferación de claves sueltas para atributos que pertenecen a una misma entidad.
- **Operaciones selectivas:** Permite modificar atributos individuales (ej. solo el estado) sin cargar todo el objeto en memoria.

Caso Aplicado:
Un robot de software actualizando su progreso: HSET auditoria:proc_101 estado 'finalizado' fecha 'hoy'.

Clase Magistral: Cuando una entidad tiene varios atributos, crear claves simples como 'usuario_estado' se vuelve inmanejable. Los Hashes nos permiten guardar bajo un mismo techo lógico todos los atributos operativos de un proceso. Y lo mejor: podemos alterar un solo campo sin mover toda la estructura.

Sets: Pertenencia y Control de Unicidad en O(1)

- **Colecciones únicas:** Ignoran y descartan matemáticamente datos duplicados en el momento de la inserción.
- **Rendimiento constante O(1):** Validar si un elemento existe toma el mismo tiempo sin importar si hay 10 o 10 millones de registros.
- **Operaciones matemáticas:** Facilitan calcular intersecciones y diferencias de conjuntos en tiempo real.



Caso Aplicado:
Verificar en microsegundos si una transacción proviene de una IP en lista negra: SISMEMBER transacciones:sospechosas 'trx_999'.

Clase Magistral: Los Sets tienen dos súper poderes: garantizan la unicidad automáticamente y su tiempo de respuesta es O(1). Tardan exactamente lo mismo en buscar en un conjunto de diez registros que en uno de diez millones. Fundamental para listas de bloqueo o prevención de fraude.

Contadores: Acumulación Atómica de Eventos



- **Operaciones atómicas (INCR/DECR)**: Suman o restan valores numéricos sin miedo a condiciones de carrera.
- **Traducción de eventos a estados**: Un simple número en Redis representa el acumulado de un comportamiento en tiempo real.
- **Disparador de reglas**: Estructura fundamental para implementar límites de uso (rate limiting) y bloqueos.

Caso Aplicado:
Monitoreo de seguridad: INCR login:intentos:user_45 para disparar un bloqueo automático si supera los 5 intentos fallidos.

Clase Magistral: Un contador en Redis no es solo aritmética; es una forma de modelar el comportamiento. Gracias a que INCR es atómico, miles de servidores pueden incrementar el contador sin colisionar. Si combinamos un contador con un límite, obtenemos un motor de reglas en tiempo real.

TTL y Ventanas Temporales: El Dato que se Limpia Solo



- **Modelado de Temporalidad:** No toda la información debe conservarse; estados momentáneos y sesiones tienen una ventana de validez.
- **Expiración Automática (EXPIRE):** El motor limpia pasiva y activamente las claves vencidas sin procesos batch nocturnos.
- **Ventanas de observación:** Permite evaluar métricas y comportamientos en intervalos exactos de tiempo.

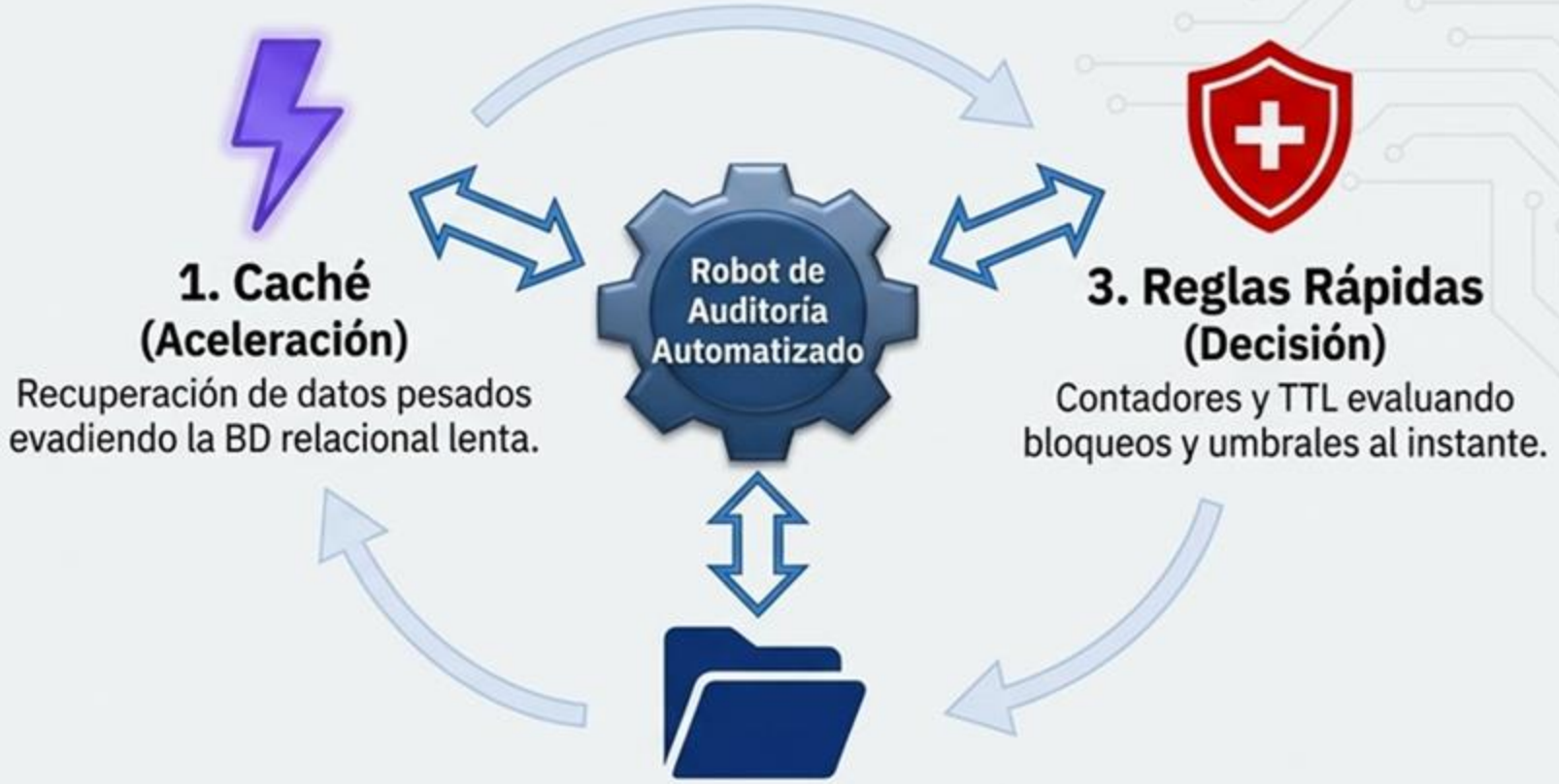
Caso Aplicado:
Bloqueo por fraude de 10 minutos:
SET login:bloqueado:u330 '1' EX 600.
El usuario se desbloquea solo, sin intervención.

Clase Magistral: En bases tradicionales, borrar registros viejos requiere pesados procesos. En Redis, le asignamos un Tiempo de Vida (TTL). Cuando el reloj llega a cero, el dato desaparece. Esto no es solo limpieza, es modelado operativo. Permite sostener estados exactamente durante la ventana de tiempo que requiere el negocio.

Arquitectura en Acción: Caché, Estado y Reglas Rápidas

- **Plano Arquitectónico:** Redis no compite con la BD relacional, la potencia asumiendo la carga operativa extrema.
- **Sinergia:** La combinación de estructuras nativas en RAM genera un ecosistema de toma de decisiones en tiempo real.

Caso Aplicado:
Un proceso cachea el reporte (Caché), registra que está "procesando" (Estado) y aborta si la IP suma errores masivos (Reglas).



1. Caché (Aceleración)
Recuperación de datos pesados evadiendo la BD relacional lenta.

3. Reglas Rápidas (Decisión)
Contadores y TTL evaluando bloqueos y umbrales al instante.

2. Gestión de Estado (Flujo)
Hashes actualizando en qué paso exacto se encuentra el robot.

Clase Magistral:
Para sintetizar: en arquitecturas modernas Redis cumple tres roles. Funciona como **Caché** para acelerar; como **gestor de Estado** para saber en qué etapa estamos; y como **motor de Reglas Rápidas** para decidir al instante. Juntando estas piezas, tenemos un plano maestro diseñado para el alto rendimiento.

Actividad

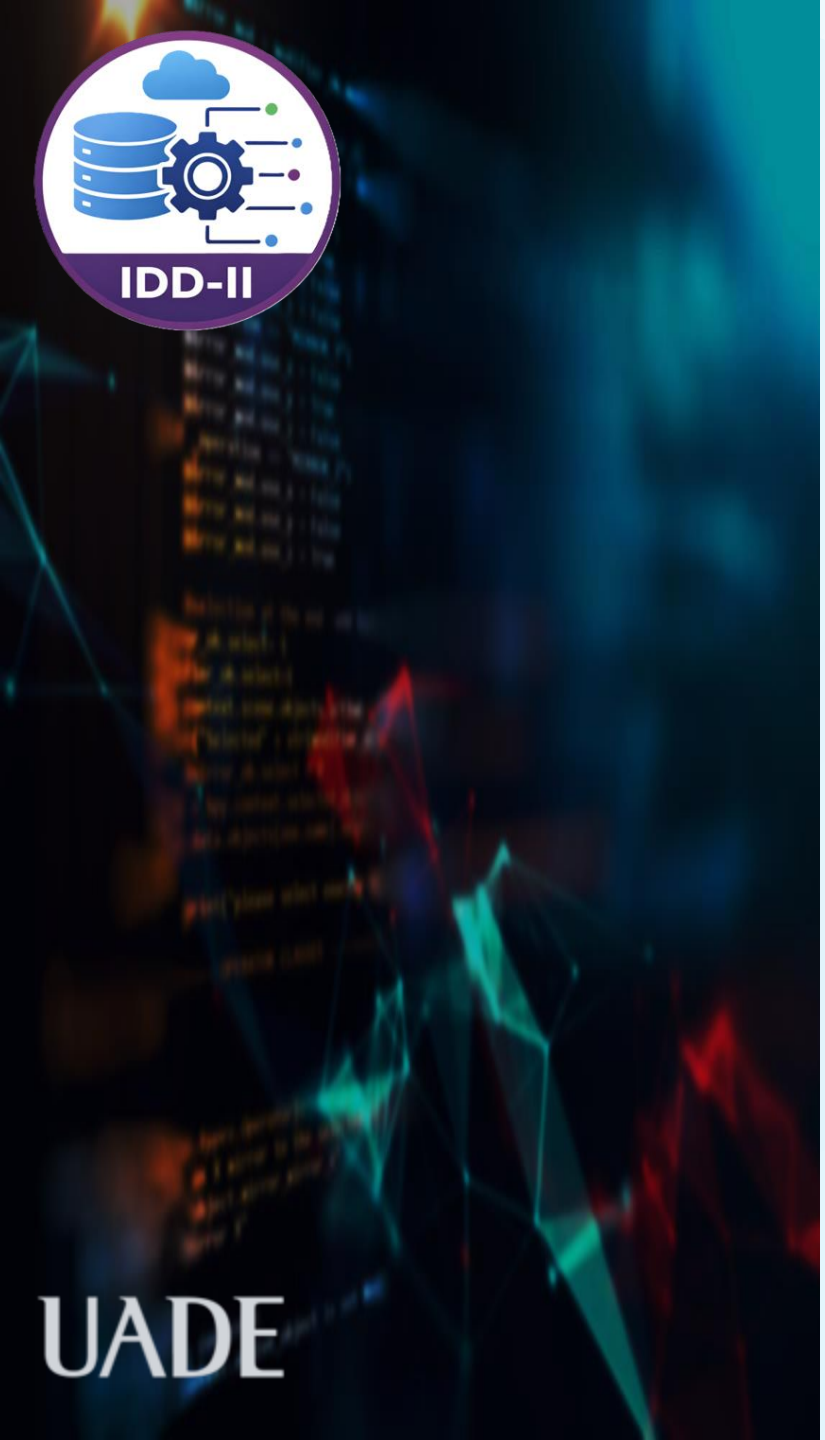
Instalación de Redis

Ir a https://docs.docker.com/desktop/setup/install/windows-install/Descargar

Docker Desktop for Windows - x86_64

UADE

2026



PROXIMA CLASE

1° Parcial

Tema:

Unidad 1: Concepto de NoSQL

Unidad 2: Distintos Modelos NoSQL

Documental – MongoDB

Grafos – Cyper/Neo4j

Clave-Valor – Redis